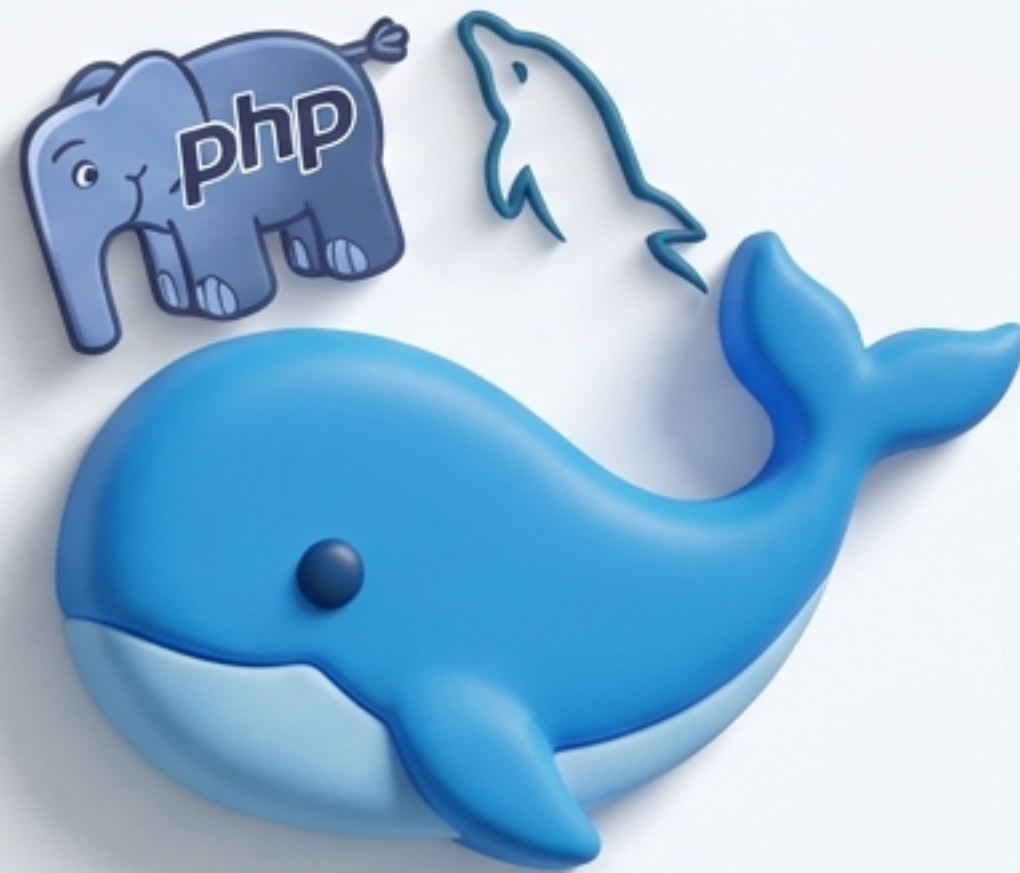


TP Docker n°3 : Déployer une application PHP + MySQL

Un guide pratique avec `docker-compose`



Nos Objectifs



Découvrir et maîtriser `docker-compose`



Automatiser le déploiement d'une stack multi-conteneurs



Comprendre la persistance des données avec les `volumes`



Distinguer `bind-mount` et `volume`



Tester la communication réseau entre conteneurs



Maîtriser le cycle de vie des services (`down` vs `down --volumes`)

Étape 1 : Préparer la Structure du Projet

**Travail à faire :

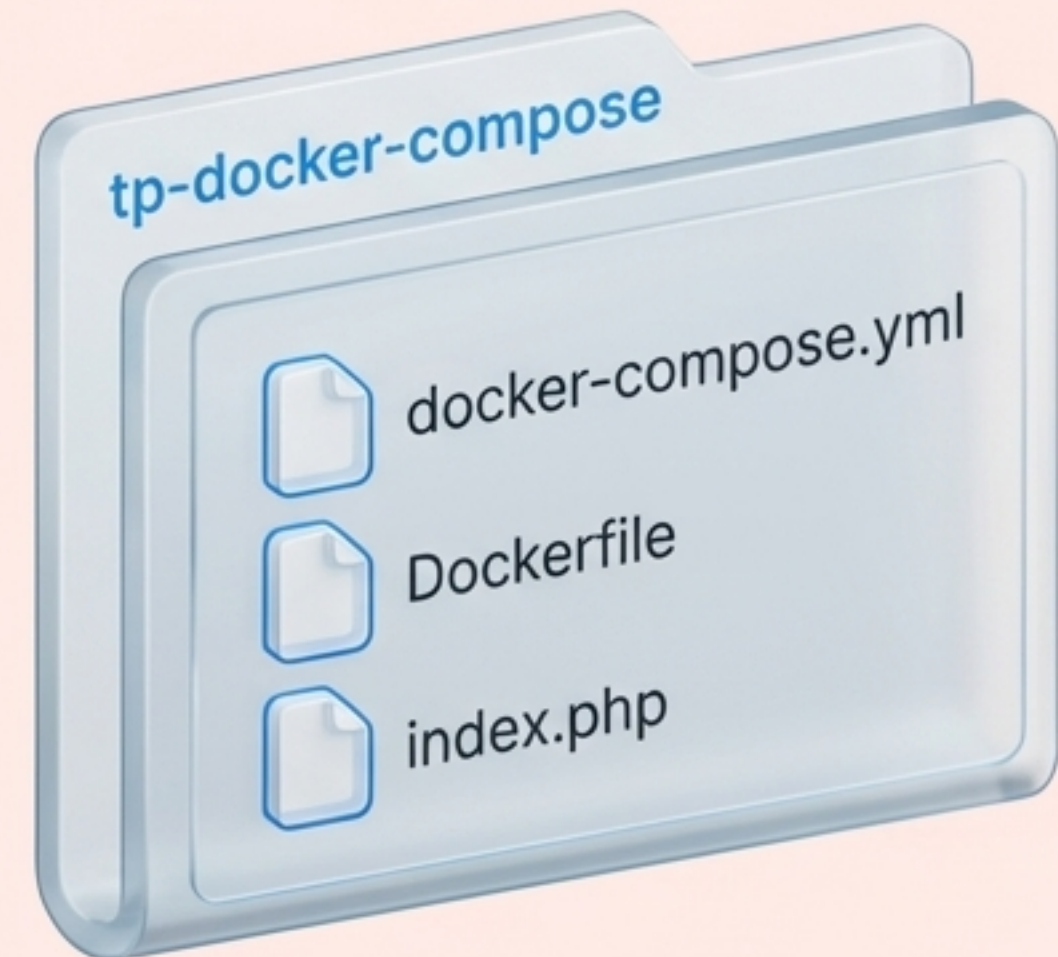
1. Ouvrez un terminal.
2. Créez le répertoire du projet avec les commandes suivantes :

>_ TERMINAL

```
mkdir tp-docker-compose  
cd tp-docker-compose
```

Information complémentaire

**Arborescence Cible :



Étape 2 : Créer le cœur de l'application (index.php)

Travail à faire : Créez le fichier index.php et copiez-y le code suivant.

```
<?php
$host = "db"; // Le nom du service MySQL dans docker-compose
$user = "root";
$pass = "rootpass";
$dbname = "demo";

$conn = new mysqli($host, $user, $pass, $dbname);

if ($conn->connect_error) {
    die("<h1>Connexion MySQL échouée 🚫 </h1>" . $conn->connect_error);
}

echo "<h1>Connexion MySQL réussie 🎉 </h1>";

$result = $conn->query("SELECT 'Hello depuis MySQL 🐙 ' AS message;");
$row = $result->fetch_assoc();
echo "<p>" . $row["message"] . "</p>";

$conn->close();
?>
```

Notez bien : l'hôte est le nom du service `db` défini dans `docker-compose.yml`, pas une adresse IP !

Étape 3 : Personnaliser notre image PHP (Dockerfile)

💡 L'image officielle php:8.2-apache ne contient pas l'extension mysqli nécessaire pour parler à MySQL, ni l'outil ping pour nos tests réseau. Nous devons donc les ajouter.

Travail à faire : Créez le fichier Dockerfile avec le contenu suivant.

```
FROM php:8.2-apache
```

```
# Installer ping pour tester la communication réseau entre services
```

```
RUN apt update && apt install -y iputils-ping
```

```
# Installer les extensions PHP nécessaires pour MySQL
```

```
RUN docker-php-ext-install mysqli pdo_mysql
```


Étape 4 : Définir l'orchestrateur (docker-compose.yml)

Travail à faire : Créez le fichier docker-compose.yml. C'est le chef d'orchestre qui définit et connecte tous nos services.

```
services:
  web:
    build: .
    container_name: php-web
    ports:
      - "8080:80"
    volumes:
      - ./var/www/html
    depends_on:
      - db
    networks:
      - tpnet

  db:
    image: mysql:8.0
    container_name: mysql-db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: demo
    volumes:
      - dbdata:/var/lib/mysql
    networks:
      - tpnet

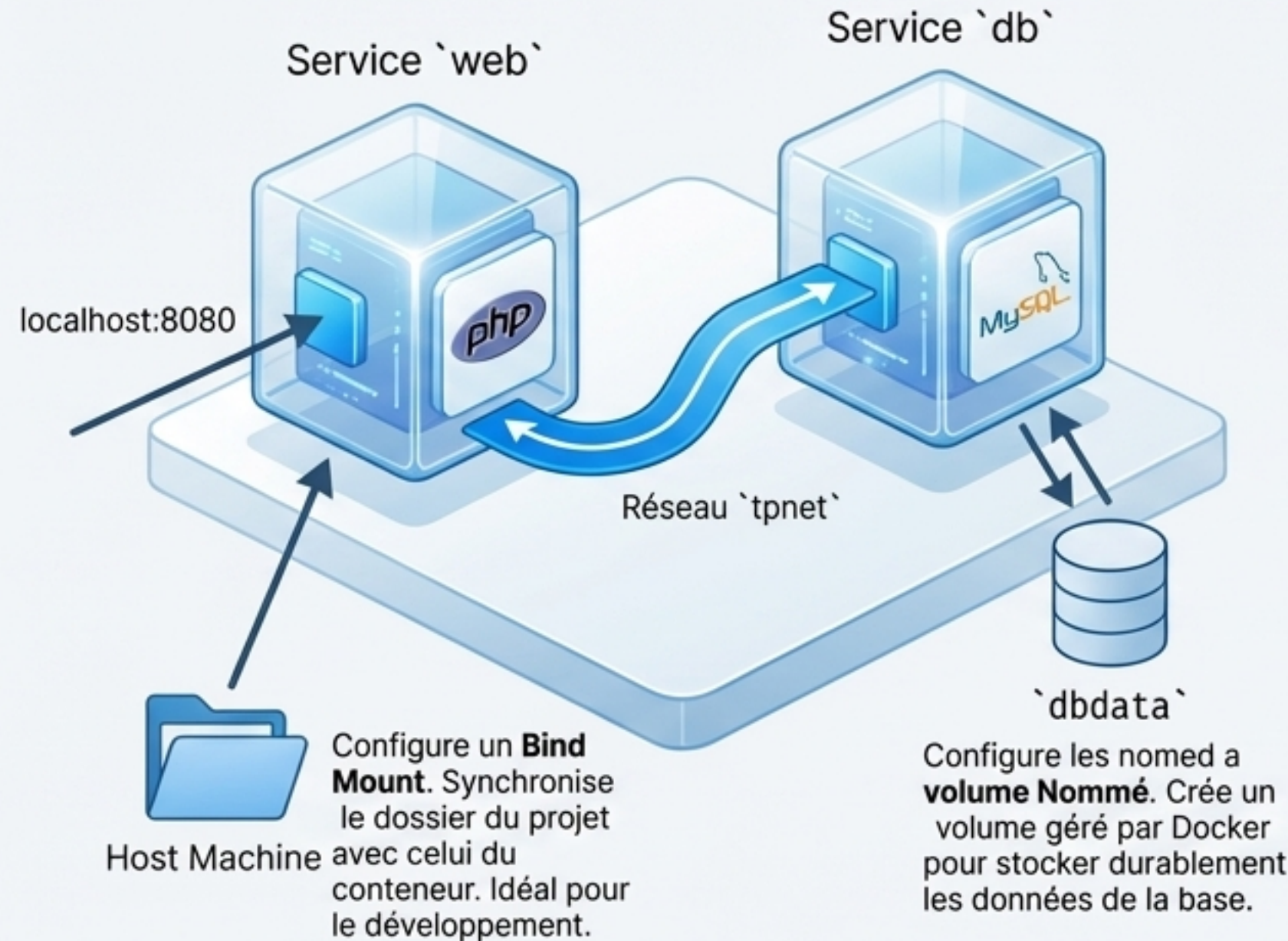
networks:
  tpnet:

volumes:
  dbdata:
```


Anatomie de notre fichier `docker-compose.yml`

Le service `web` (Notre PHP)

- `build: .`: Construit une image à partir du `Dockerfile` local.
- `ports: "8080:80"`
Mappe le port 8080 de notre machine (hôte) au port 80 du conteneur.
- `volumes: ./var/www/html`
Bind Mount. Synchronise le dossier du projet avec celui du conteneur. Idéal pour le développement.
- `depends_on: db`
S'assure que le service `db` démarre avant le service `web`.



Le service `db` (Notre MySQL)

- `image: mysql:8.0`
Utilise une image officielle depuis Docker Hub.
- `environment: ...`
Configure les variables d'environnement pour initialiser MySQL.
- `volumes:`
`dbdata:/var/lib/mysql`
Volume Nommé. Crée un volume géré par Docker pour stocker durablement les données de la base.

Étape 5 : Construire et Lancer la Stack

Travail à faire : Depuis votre terminal, à la racine du projet, exécutez les commandes suivantes.

1. Construit notre image PHP personnalisée (si modifiée)

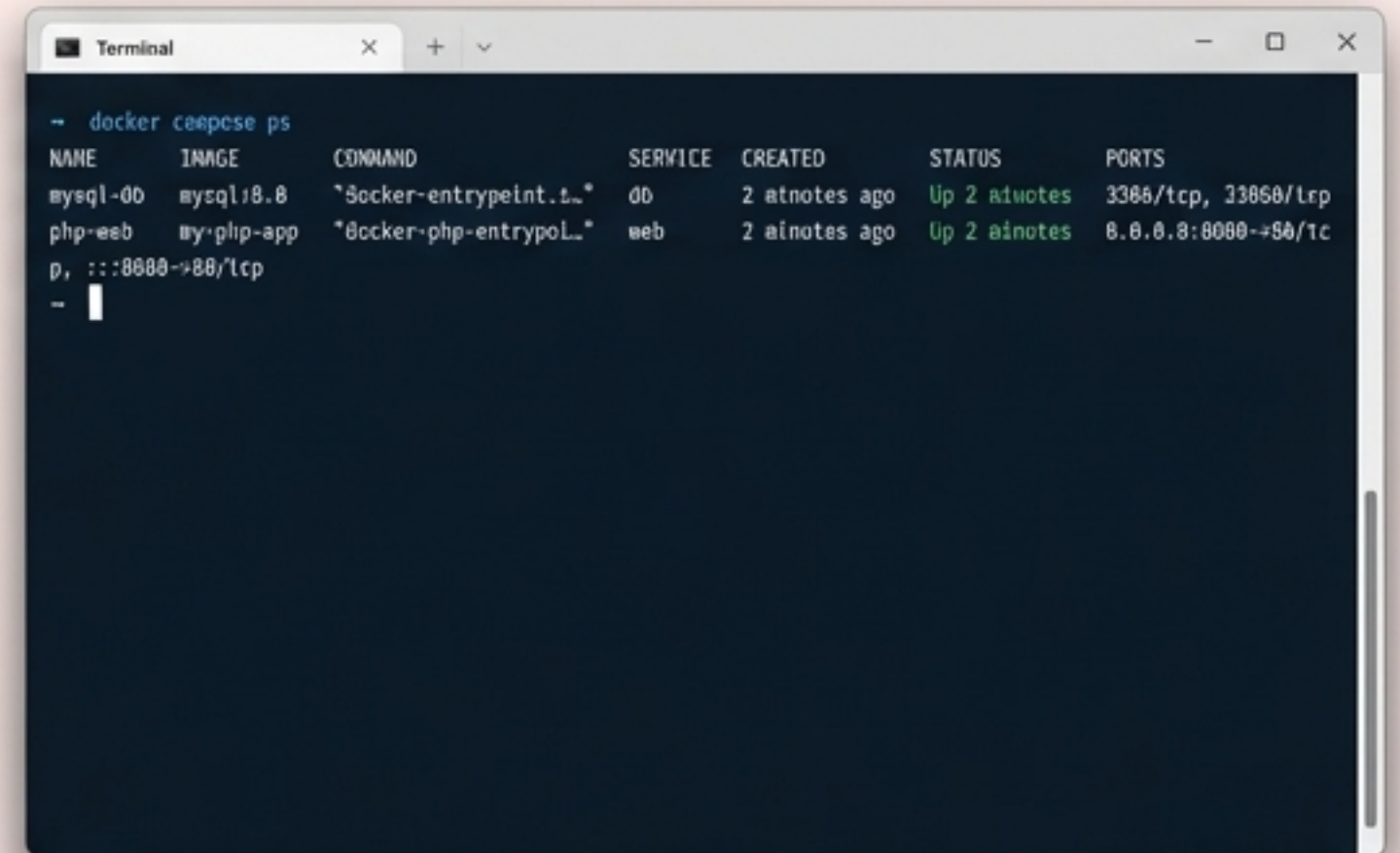
```
docker compose build
```

2. Démarre les services en arrière-plan (-d pour "detached")

```
docker compose up -d
```

3. Vérifie que les deux conteneurs tournent correctement

```
docker compose ps
```

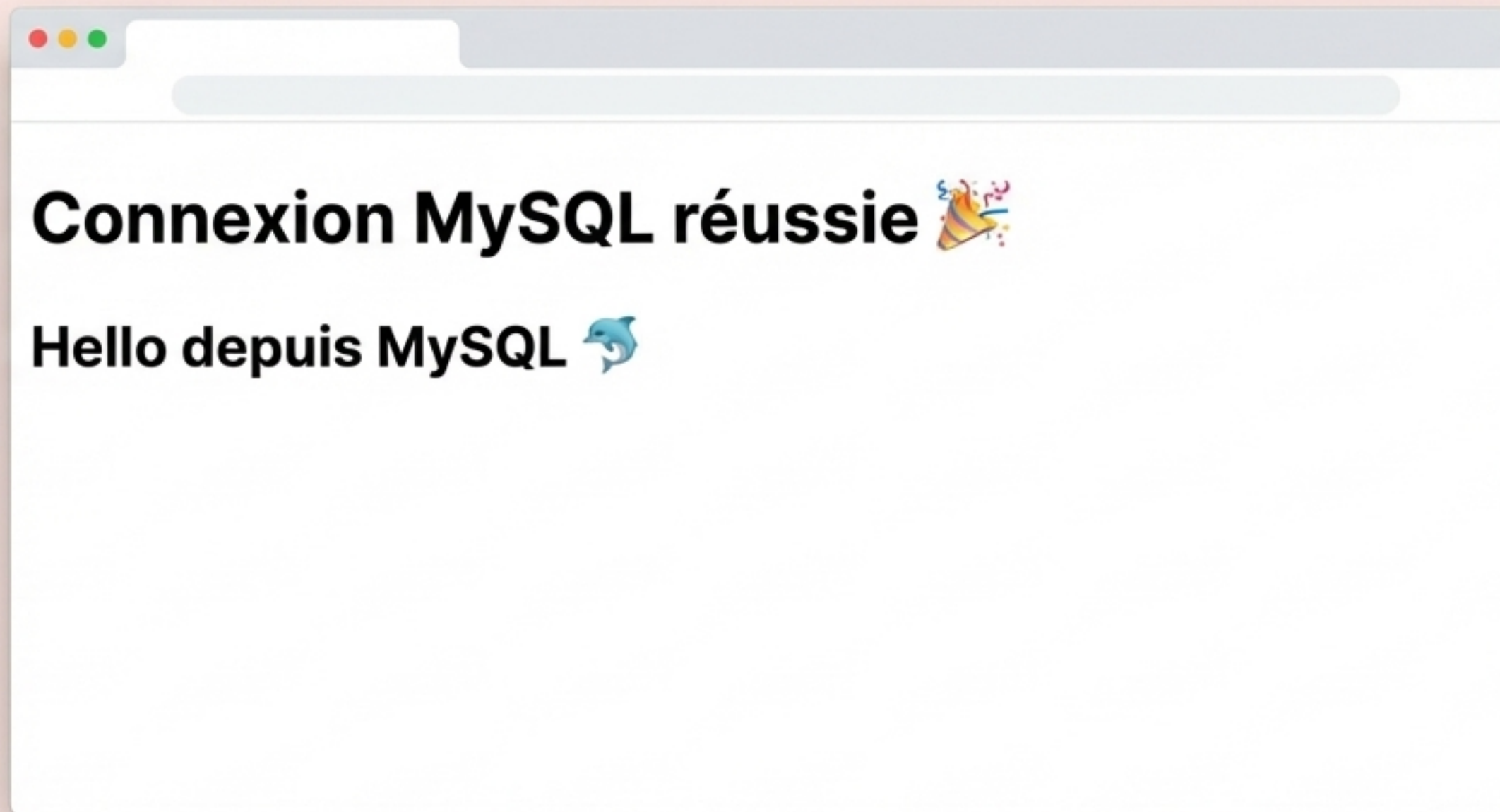


```
Terminal
~ docker compose ps
NAME          IMAGE          COMMAND                  SERVICE  CREATED      STATUS      PORTS
mysql-00      mysql:8.0      "docker-entrypoint.sh"   db        2 minutes ago Up 2 minutes 3306/tcp, 33060/tcp
php-web       my-php-app     "docker-php-entrypoint"  web       2 minutes ago Up 2 minutes 0.0.0.0:8080->80/tcp
```


Étape 6 : Vérifier le résultat dans le navigateur

Travail à faire : Ouvrez votre navigateur web et visitez l'adresse suivante :

`http://localhost:8080`



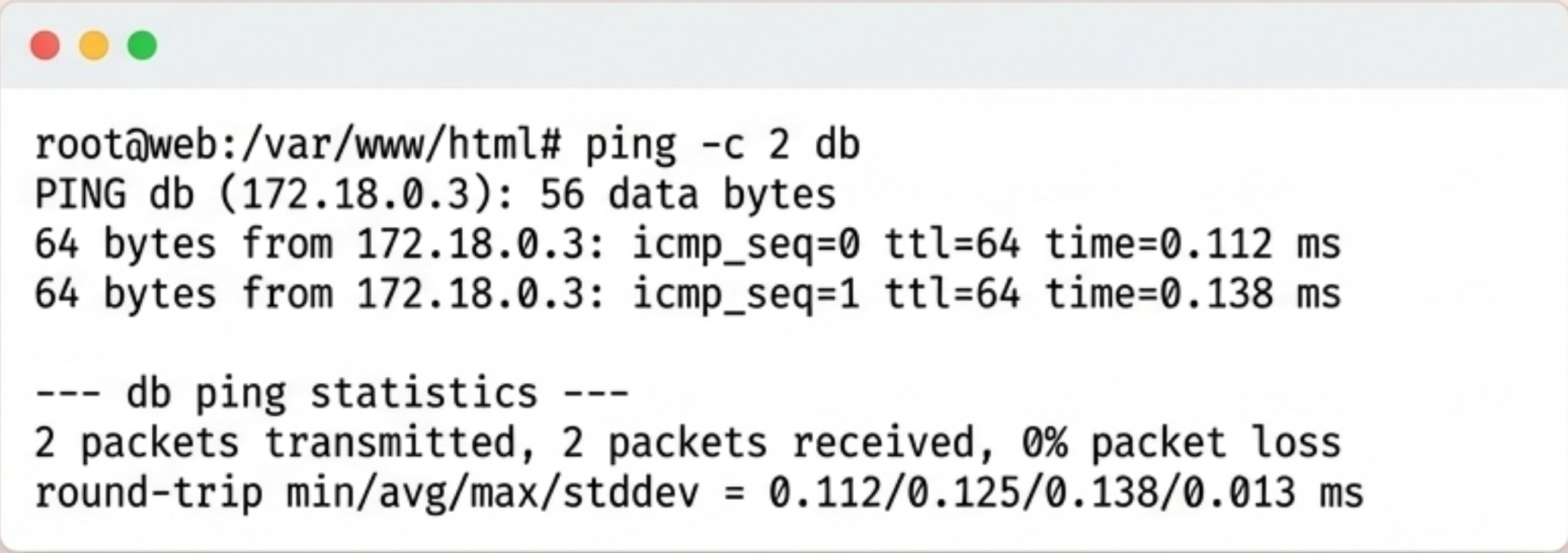
Étape 7 : Tester la communication interne des conteneurs

Nous allons vérifier que notre conteneur `web` peut communiquer avec le conteneur `db` en utilisant son nom de service, `db`, sur le réseau privé que Docker a créé.

Travail à faire :

```
# 1. Ouvrir un shell à l'intérieur du conteneur "web"  
docker compose exec web bash
```

```
# 2. Une fois dans le conteneur, pinger le service "db"  
ping -c 2 db
```

A terminal window with a light blue title bar and three colored window control buttons (red, yellow, green) on the left. The terminal text shows a successful ping from the 'web' container to the 'db' container.

```
root@web:/var/www/html# ping -c 2 db  
PING db (172.18.0.3): 56 data bytes  
64 bytes from 172.18.0.3: icmp_seq=0 ttl=64 time=0.112 ms  
64 bytes from 172.18.0.3: icmp_seq=1 ttl=64 time=0.138 ms  
  
--- db ping statistics ---  
2 packets transmitted, 2 packets received, 0% packet loss  
round-trip min/avg/max/stddev = 0.112/0.125/0.138/0.013 ms
```



C'est magique ! Docker résout automatiquement le nom `db` en l'adresse IP interne du conteneur MySQL.

Étape 8 : Inspecter le Volume de Données

Où sont stockées les données de notre base MySQL ? Pas **dans le conteneur** (qui est éphémère), mais dans un **volume** géré par **Docker** sur notre machine. Allons le trouver.

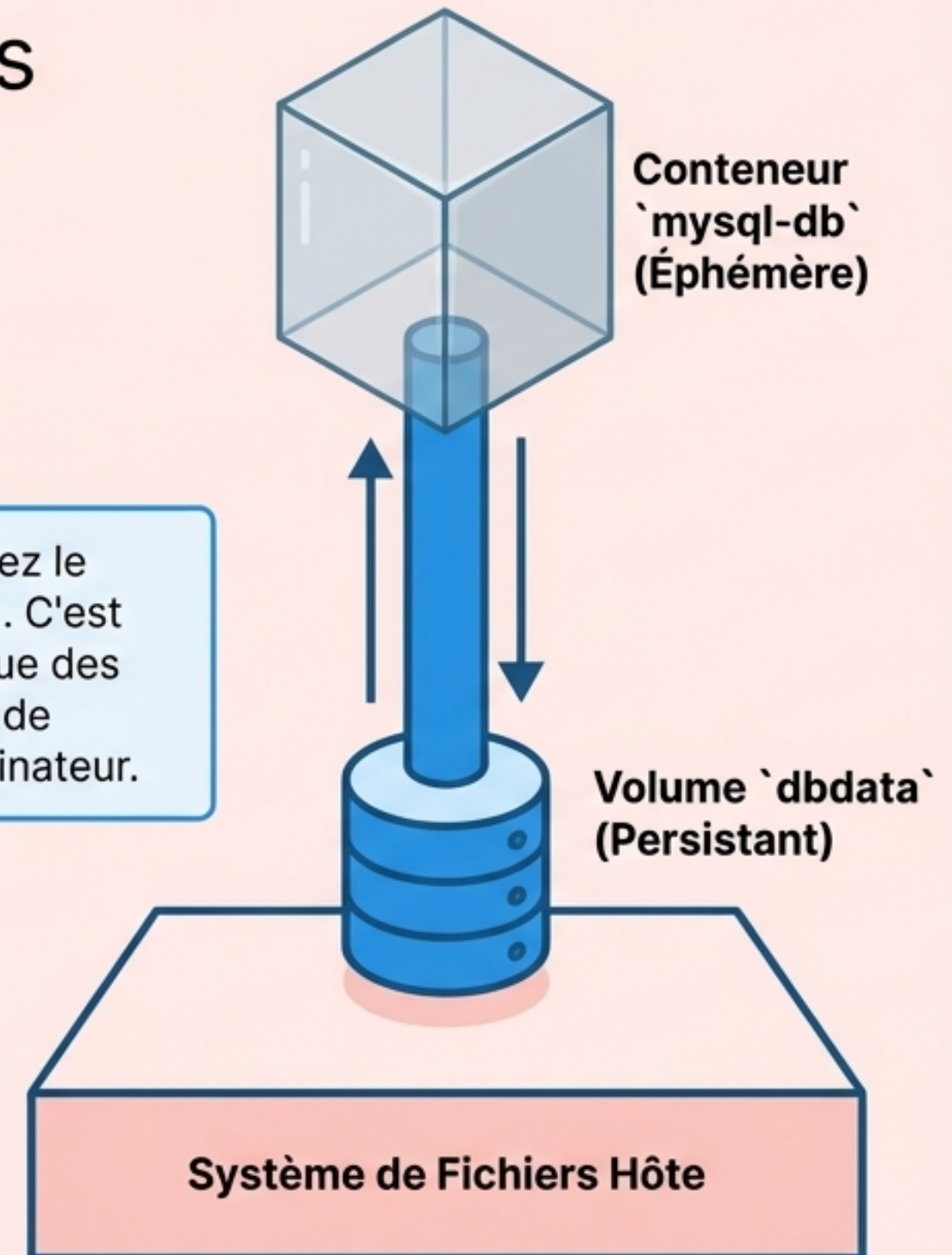
**Travail à faire :

```
# 1. Lister tous les volumes gérés par Docker
docker volume ls

# 2. Inspecter les détails de notre volume spécifique
# (Le nom est préfixé par le nom du dossier projet)
docker volume inspect tp-docker-compose_dbdata
```



Dans le résultat, repérez le champ `""Mountpoint""`. C'est l'emplacement physique des fichiers de votre base de données sur votre ordinateur.



Étape 9 & 10 : Comprendre le Cycle de Vie (`down` vs `down --volumes`)

Scénario 1 : Arrêt simple

```
docker compose down
```

Arrête et supprime les **conteneurs** et le **réseau**.

Le volume `dbdata` **N'EST PAS SUPPRIMÉ**. Vos données MySQL sont en sécurité et seront réutilisées au prochain `docker compose up -d`.



Scénario 2 : Arrêt complet

```
docker compose down --volumes
```

Arrête et supprime les **conteneurs**, le **réseau** ET les **volumes nommés** associés.

⚠ Les données du volume `dbdata` sont **DÉFINITIVEMENT SUPPRIMÉES**.



✓ Checklist du Succès : Avez-vous tout accompli ?

- ✓ Le fichier `index.php` a été créé et fonctionne.
- ✓ Le `Dockerfile` a été créé pour ajouter `ping` et `mysqli`.
- ✓ Le fichier `docker-compose.yml` a été créé et compris.
- ✓ La commande `docker compose up -d` a lancé les services avec succès.
- ✓ La page `http://localhost:8080` affiche le message de connexion réussie.
- ✓ La commande `ping db` a fonctionné depuis le conteneur `web`.
- ✓ La différence fondamentale entre `down` et `down --volumes` est comprise.